

OTP & Email Testing

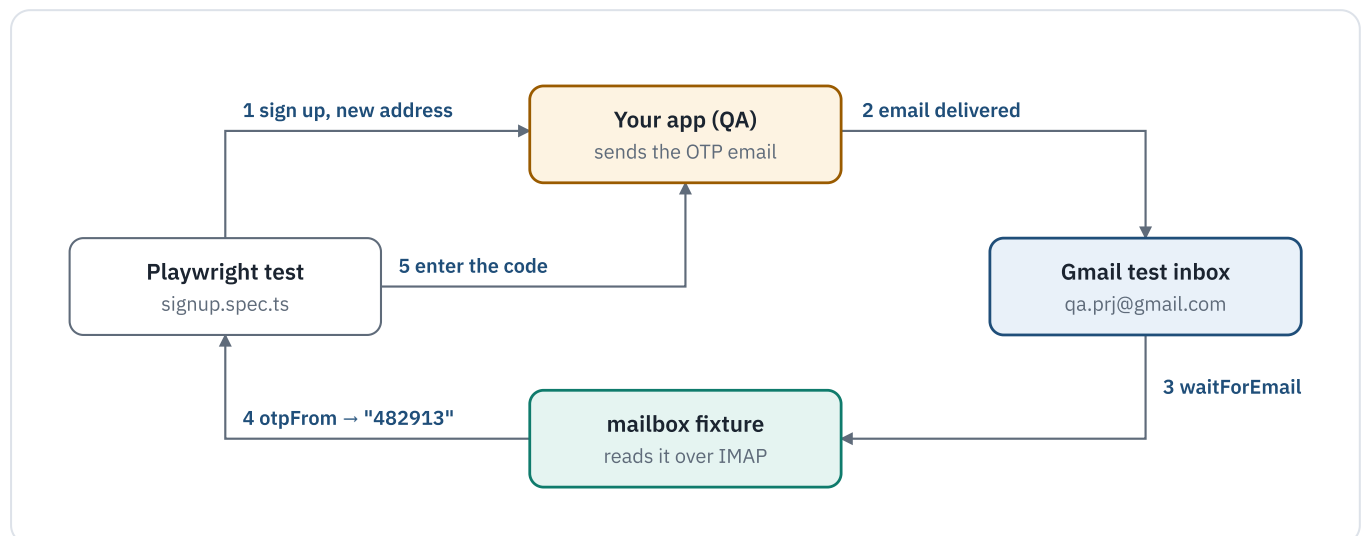
How to check that an OTP or verification email arrives, read the code from it, and test that the app accepts the right code and rejects wrong, old, reused and expired ones. Everything runs against a real test inbox, with no hard-coded codes.

TEST INBOX	IN TESTS	STANDARD CASES
One dedicated Gmail per project	The <code>mailbox</code> fixture	<code>templates/otp-testcases.md</code>

THE IDEA

How it works

The test signs up with a unique address that lands in the team's Gmail test inbox. The `mailbox` fixture reads that inbox, takes the code out of the email, and the test types it in, just like a person would.



After the test, its emails are moved to Trash automatically, so the inbox stays clean.

DECISION

Why a Gmail test inbox, not Yopmail

Yopmail is handy for a quick look by hand, but it isn't safe or reliable for automated tests.

Gmail test inbox

- ✓ Private: only the team can read it
- ✓ Read over IMAP: fast and stable, no website to click through
- ✓ Unlimited unique addresses with `+` (plus addressing)
- ✓ Already built into the framework (`mailbox`)
- ✓ Free

Yopmail

- Public: anyone who guesses the address reads the OTPs
- No API; tests would have to click through its website
- Ads, captchas and rate limits make tests flaky
- Many apps block disposable email domains

Yopmail for manual checks only

Fine for a quick manual look with throwaway data. Never for accounts or data you'd mind others seeing.

ONCE PER PROJECT

Set up the test inbox

The QA lead does this once and shares the settings with the team through the password manager.

1 Create a dedicated Gmail account

For example `qa.prj.automation@gmail.com`. Never a personal mailbox: tests read and delete emails in it.

2 Turn on 2-Step Verification

Google Account → Security → 2-Step Verification. Google requires it before it allows app passwords.

3 Create an app password

4

Google Account → search "App passwords" → name it "QA automation" → copy the 16-character password. This is what the framework uses, never the normal password.

APPS/<APP>/**.ENV** (NEVER COMMITTED)

```
MAIL_USER=qa.prj.automation@gmail.com
MAIL_PASSWORD=abcd efgh ijkl mnop
# optional:
MAIL_WAIT_SECONDS=60                # how long to wait for an
email
MAIL_IMAP_HOST=imap.gmail.com        # Outlook / Microsoft 365:
outlook.office365.com
```

5

Check the connection

```
npm run mail:check
```

Logs in, lists the folders the tests will search (inbox, spam, All Mail) and shows a sample test address. It never prints the password.

ONCE PER PROJECT

Unique addresses with plus addressing

Gmail delivers `name+anything@gmail.com` to `name@gmail.com`. Every test gets its own address, so tests never read each other's emails, even when they run in parallel.

```
qa.prj.automation+qa-auto-signup-k3x9@gmail.com
```

mailbox name **test data prefix** (TEST_DATA_PREFIX) **label** from `newAddress('signup')` **random part**

If the app rejects "+" in email addresses

Some sign-up forms reject the `+` sign. Ask the developers to allow it (it is a valid address), at least on QA. Until then, those cases can't use unique addresses.

WRITE TESTS

Code patterns

Claude writes these for you with `/qa-automate` ; this is what to expect when you review them. The example is a sign-up page with an OTP step.

Email arrives and the right code is accepted

```
test.use({ role: null }); // sign-up starts logged out

test('TC-SIGNUP-04 | Correct OTP is accepted', { tag: ['@smoke', '@TC-SIGNUP-04'] }, async ({ signupPage, otpPage, mailbox }) => {
  const email = mailbox.newAddress('signup');
  await signupPage.register({ ...newUser, email });

  const message = await mailbox.waitForEmail(email, { subject:
'verification code' });
  await otpPage.enter(await mailbox.otpFrom(message));

  await otpPage.expectVerified();
});
```

Wrong code is rejected

```
const code = await mailbox.otpFrom(await mailbox.waitForEmail(email, {
subject: 'verification code' }));
const wrong = code.slice(0, -1) + ((Number(code.at(-1)) + 1) % 10); //
same length, one digit changed
await otpPage.enter(wrong);
await otpPage.expectError('Invalid code. Please try again.');
```

// exact text from the requirement

Resend: the old code stops working, the new one works

```
const first = await mailbox.waitForEmail(email, { subject: 'verification
code' });
const firstCode = await mailbox.otpFrom(first);

await otpPage.resend();
const second = await mailbox.waitForEmail(email, { subject:
'verification code', newerThan: first });
const secondCode = await mailbox.otpFrom(second);
```

```
await otpPage.enter(firstCode);
await otpPage.expectError('This code has expired. Request a new one. ');
await otpPage.enter(secondCode);
await otpPage.expectVerified();
```

Email content

```
expect(message.from).toContain('PRJ');
expect(message.subject).toBe('Your PRJ verification code');
expect(message.text).toContain(`Hi ${newUser.firstName}`);
expect(message.text).not.toMatch(/{{|}}|\bundefined\b|\bnull\b/); //
no broken template
```

Verification link instead of a code

```
const message = await mailbox.waitForEmail(email, { subject: 'Confirm
your email' });
await page.goto(await mailbox.linkFrom(message, '/verify')); // picks
the verify link, not "unsubscribe"
```

Codes that aren't 6 digits

```
await mailbox.otpFrom(message, { length: 4 }); //
4-digit PIN
await mailbox.otpFrom(message, { pattern: /code: ([A-Z0-9]{8})/ }); //
letters, dashes: the code in group 1
```

The email is in the report

`waitForEmail` attaches every email it finds (sender, subject, text) to the Allure report, so a failure shows exactly what arrived.

Never hard-code an OTP

Always read it from the email. The only exception: a fixed QA-only code the developers configured on the QA environment, read with `requireEnv('OTP_CODE')`, never written in a test file.

What to test

The full list with steps, expected results and how to automate each one is in `templates/otp-testcases.md`. Copy the rows that apply into the module's test-cases file and fill in the exact messages from the requirement.

TC-...-01 to 03

Delivery

- The email arrives within the agreed time
- Right sender, subject, greeting, expiry text
- No broken template placeholders
- The code has the expected length and format

TC-...-04 to 09

Valid and invalid input

- Correct code accepted
- Wrong code rejected, with the exact message
- Empty, too short or too long, letters
- Pasted code with spaces

TC-...-10 to 14

Code lifecycle

- Old code rejected after resend
- Resend sends a new code; resend is limited
- A code works only once
- Expired code rejected

TC-...-15 to 20

Security, accessibility, links

- Too many wrong attempts lock the code
- Another account's code rejected
- Code never in the URL or API response
- OTP screen accessible; verify links work once

Claude uses the template too

When a requirement has an OTP or email step, `/qa-testcases` covers these cases automatically. Review them like any other test cases.

Expiry and limits, without waiting

An OTP that expires after 10 minutes can't be tested by waiting 10 minutes: the framework forbids sleeps, and the suite would crawl. Ask the developers for one of

these on the QA environment.

OPTION	WHAT THE DEVELOPERS PROVIDE	THEN
Short expiry on QA	OTP expiry configurable, e.g. 30 seconds on QA	Automate the expired case with a web-first wait for the "expired" state
Lower limits on QA	Resend cooldown and max attempts configurable	Automate resend-limit and lock cases quickly
Test hook	A QA-only API to expire a code or reset attempts	Call it with <code>api</code> in the test, no waiting at all
None of these	Nothing	Keep the case <code>Automate: later</code> and test it by hand each release

PRACTICE

Try the demo

The example project has a demo "app" that sends real OTP emails to your test inbox and checks codes like a real verify screen. Nine working tests show every pattern above.

```
npm run mail:check      # the saucedemo .env needs MAIL_USER /  
MAIL_PASSWORD  
npm run demo:email      # runs TC-MAILDEMO-01 to 09  
npm run report
```

TEST	SHOWS
TC-MAILDEMO-01	Email arrives with the right content
TC-MAILDEMO-02	Reading the code, not the other numbers in the email
TC-MAILDEMO-03	Resend: the newest code is read (<code>newerThan</code>)
TC-MAILDEMO-04	Reading a verification link

TEST	SHOWS
TC-MAILDEMO-05	Correct code accepted
TC-MAILDEMO-06	Wrong code rejected
TC-MAILDEMO-07	Old code rejected after resend
TC-MAILDEMO-08	Code works only once

TC-MAILDEMO-09 (0-second QA expiry)
 Files: `apps/saucedemo/tests/api/demo/email-demo.spec.ts` and `apps/saucedemo/demo/demoAppMailer.ts`. The demo tests are tagged `@demo` and never run in normal suites.

HELP

Quick reference: the mailbox fixture

CALL	WHAT IT DOES
<code>mailbox.newAddress('sign up')</code>	A new unique address that arrives in the test inbox
<code>mailbox.waitForEmail(address, { subject })</code>	Waits (up to <code>MAIL_WAIT_SECONDS</code>) for the newest email to that address; checks inbox, spam and All Mail; attaches it to the report
<code>{ newerThan: first }</code>	Only an email that arrived after <code>first</code> (resend)
<code>{ timeoutSeconds: 120 }</code>	A longer wait for one slow email
<code>mailbox.otpFrom(email)</code>	The 6-digit code next to a word like "code" or "OTP"; <code>{ length }</code> or <code>{ pattern }</code> for other formats
<code>mailbox.linkFrom(email, '/verify')</code>	The first link whose address contains <code>/verify</code>

CALL	WHAT IT DOES
HELP (automatic)	After each test, its emails are moved to Trash

Common problems

mail:check : "Login FAILED"

`MAIL_PASSWORD` must be the 16-character app password, not the normal password, and 2-Step Verification must be on. Check `MAIL_USER` is the full address.

"No email to ... arrived within 60 s"

Check the address the app actually received (the attached steps show it), and that the `subject` filter matches words in the real subject. If the app is just slow, raise `MAIL_WAIT_SECONDS` or pass `{ timeoutSeconds }`. Spam is already searched.

"Several 6-digit numbers in email ..."

The email has other numbers (order IDs, phone numbers) and none next to a word like "code". Pass `{ pattern: /Your code is (\d{6})/ }` matching the real wording.

The test read the old code after resend

Use `{ newerThan: first }` for the second email.

The sign-up form rejects the address

The form doesn't accept `+`. See [Unique addresses](#): ask the developers to allow it on QA.

Email tests fail only when many run at once

Gmail limits simultaneous IMAP connections (about 15 per account). Run email-heavy suites with fewer workers, e.g. `--workers=4`.

Related: [QA Team Setup](#) · [Sprint QA Process](#) · test cases: `templates/otp-testcases.md` · rules: `CLAUDE.md` (email flows).

